

CarbonServe: Uncertainty-Aware Carbon Routing for SLO-Constrained LLM Inference Across Geo-Distributed Cloud Regions

Ravani Roshan

Independent Research

ravaniroshansingh@gmail.com

Abstract—Large language model (LLM) inference has become a dominant operational workload in cloud platforms, and its electricity consumption translates directly into operational carbon emissions that vary by up to an order of magnitude across cloud regions and hours. Existing carbon-aware routing proposals for LLM serving act on point forecasts of grid carbon intensity (CI) and are evaluated only in simulation; recent results in carbon-aware computing show that such point-forecast decisions can backfire and increase emissions when forecasts err. We present CarbonServe, an uncertainty- and SLO-aware request routing layer for geo-distributed LLM inference. CarbonServe combines a day-ahead persistence CI forecaster with per-grid empirical error distributions, and offloads traffic to a remote region only when the remote grid’s pessimistic (upper) CI band beats the home grid’s optimistic (lower) band by a margin, subject to utilization guards that protect latency SLOs. We evaluate CarbonServe on 44.1 million production LLM inference requests (Azure traces) replayed against real hourly CI data for 11 grids in North America and Europe. Under realistic forecast error (10.1% MAPE), CarbonServe reduces operational carbon by 29.4% versus locality-only serving—capturing 96% of the perfect-information oracle’s savings—while reducing routing backfire (requests directed into grids dirtier than home) by 27× relative to CASPER-style point-forecast routing. As forecast quality degrades, point-forecast policies lose up to half their savings and incur order-of-magnitude emission penalties, whereas CarbonServe degrades gracefully and dominates on both savings and risk. All latency SLOs (p95 TTFT, p99 TPOT) are maintained with negligible router overhead.

Index Terms—cloud computing, carbon-aware scheduling, large language models, LLM inference, sustainable computing, geo-distributed systems, service level objectives

I. Introduction

The inference phase of large language models (LLMs) has become one of the largest and fastest-growing consumers of cloud compute. Inference workloads are continuous, user-facing, and bound by strict latency service level objectives (SLOs) such as time-to-first-token (TTFT) and time-per-output-token (TPOT), and they run on power-hungry GPUs whose energy consumption translates directly into operational carbon emissions. Recent estimates indicate that the annual operating cost of serving a popular LLM can exceed its one-time training cost by a factor of 25 or more, and that the cumulative operational carbon of inference now dominates that of training [12].

At the same time, the carbon intensity (CI) of electricity—the grams of CO₂-equivalent emitted per kilowatt-hour—varies dramatically in both space and time. Across the 11 electricity grids we study in this paper, hourly CI during a representative week spans roughly 44 to 478 gCO₂/kWh, and the relative ranking of grids changes from hour to hour as

renewable generation and demand fluctuate. A cloud platform operating inference capacity in multiple regions therefore faces a compelling opportunity: route each request to a region whose grid is currently cleanest, subject to serving capacity and latency SLOs.

A growing line of work has begun to explore this opportunity, proposing carbon-aware scheduling and provisioning for LLM inference [11]–[16]. However, these proposals share two important limitations. First, they act on **point forecasts** of carbon intensity, even though it has been shown in the broader carbon-aware computing literature that load-shifting decisions based on point CI predictions can misfire: when the forecast is wrong, work is moved into a dirtier grid and emissions increase rather than decrease [10]. Second, existing carbon-aware LLM serving evaluations are exclusively trace-driven simulations without risk analysis—no prior system quantifies how often carbon-motivated routing decisions actually backfire, or bounds the damage when they do.

This paper addresses both gaps. We present CarbonServe, a routing layer for geo-distributed LLM inference that treats carbon-intensity forecast **uncertainty** as a first-class design constraint. CarbonServe builds a day-ahead CI forecast with per-grid empirical error distributions derived from two years of historical data, forms a 90% confidence band around each grid’s forecast, and routes traffic away from a request’s home region only when a candidate region’s pessimistic CI bound is better than the home region’s optimistic bound by a configurable margin. This confidence rule guarantees, with high probability, that offloaded work lands in a genuinely cleaner grid. A utilization guard caps the load admitted to any destination region so that latency SLOs are preserved under offloading.

We evaluate CarbonServe by replaying 44.1 million real production LLM inference requests from the Azure LLM inference traces [1], [22]—covering two services (a conversational assistant and a coding assistant) over a full week—against real hourly carbon intensity data for 11 grids in North America and Europe [20], [21]. Our evaluation quantifies both average carbon savings and routing backfire risk, measured as the rate and emission penalty of requests routed into grids whose true CI was worse than the home grid’s. The contributions of this paper are:

- A characterization of the carbon-savings opportunity for geo-distributed LLM inference on real production traces and real grid data, showing that oracle carbon-aware routing could reduce operational emissions by 30.7% (con-

versational service) and 18.9% (coding service) without violating latency SLOs.

- CarbonServe, an uncertainty-aware, SLO-safe carbon routing policy that uses empirical CI forecast error bands and a confidence rule to bound routing backfire, together with utilization guards that protect SLOs.
- A risk-aware evaluation methodology for carbon-aware LLM serving, introducing misroute rate and misroute penalty as metrics of decision backfire, alongside carbon, energy, and SLO metrics.
- An empirical demonstration that under realistic forecast error (10.1% MAPE day-ahead), CarbonServe attains 96% of the oracle’s savings while reducing the backfire penalty by 27× relative to CASPER-style point-forecast routing; and that under degraded forecast quality CarbonServe dominates point-forecast policies on both savings and risk.

All experiments, datasets, and code paths in this paper are reproducible from public data sources.

II. Background and Motivation

II.I. LLM Inference Serving and SLOs

Modern LLM serving engines such as vLLM [18] and DistServe [19] process requests in two phases with distinct computational profiles: a compute-bound **prefill** phase that ingests the prompt, and a memory-bound **decode** phase that generates tokens autoregressively. Interactive deployments enforce SLOs on both phases—commonly a p95 or p99 bound on TTFT (e.g., 2 s) and on per-token latency TPOT (e.g., 150 ms). A geo-distributed deployment serves requests from the region nearest the user to minimize network latency; requests may, in principle, be routed to any region that hosts the model, at the cost of added wide-area latency and reduced data locality.

II.II. Spatial and Temporal Carbon Intensity Diversity

Fig. 1 shows the real hourly carbon intensity of the 11 grids used in this study over the simulated week. Two properties create the routing opportunity. First, the spread is large: weekly mean CI ranges from roughly 44 gCO₂/kWh (SE) to 478 gCO₂/kWh (PL), an order of magnitude. Second, the ranking is non-stationary: the cleanest grid changes across hours as solar, wind, and demand patterns shift, so a static assignment of work to the historically cleanest region leaves savings on the table.

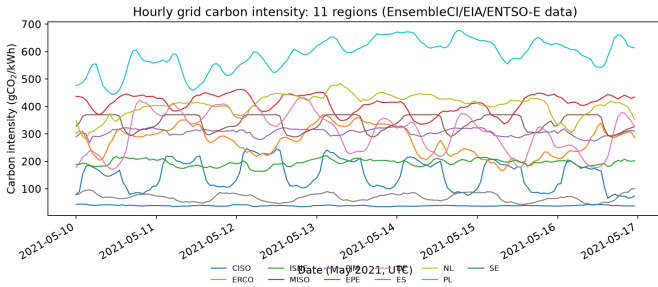


Fig. 1. Real hourly grid carbon intensity for the 11 study grids over the simulated week (EnsembleCI datasets, derived from EIA and ENTSO-E data [20], [21]).

II.III. The Backfire Risk of Point-Forecast Carbon Routing

Carbon intensity forecasts are necessary because routing decisions must be made ahead of the energy consumption they govern. The simplest practical forecaster is day-ahead persistence—predicting $CI(t)$ from $CI(t-24\text{ h})$ —which achieves a mean absolute percentage error of 10.1% across our 11 grids. Prior work on datacenter decarbonization has shown that acting on point forecasts without uncertainty quantification can increase emissions by 2.7%–14% when predictions err, because work migrates into grids that are actually dirtier than the source [10]. Whether this phenomenon matters for LLM serving—where routing is fine-grained, latency-constrained, and capacity-guarded—has not previously been quantified. Section VI shows that it does, severely: at 4× degraded forecast quality, 22.8% of offloaded traffic under a CASPER-style policy lands in grids dirtier than home, at an avoidable emission penalty of 13.4 kg CO₂e for a single week of conversational traffic.

III. Related Work

III.I. Carbon-Aware Cloud Computing

Carbon-aware scheduling exploits the spatiotemporal flexibility of workloads to align energy use with clean electricity. Google’s production system shifts flexible batch jobs temporally [7]; CarbonScaler scales elastic batch jobs with carbon intensity [4]; CASPER provides carbon-aware provisioning and load balancing for geo-distributed web services under SLOs [5]; and Carbon Explorer analyzes design trade-offs for 24/7 carbon-free datacenters [6]. Studies of the limits of carbon-aware shifting [8] and of the sunk-carbon fallacy in attribution [9] caution that naive shifting can be counterproductive—an insight our results confirm and quantify for LLM serving. None of these systems targets the fine-grained, latency-sensitive, GPU-dense characteristics of LLM inference, and none incorporates CI forecast uncertainty into routing decisions.

III.II. Energy-Efficient LLM Serving

A rich line of systems reduces the energy of LLM inference within a single cluster: DynamoLLM co-optimizes instance count, model parallelism, and GPU frequency under SLOs [1]; Splitwise disaggregates prefill and decode for efficiency [2]; TAPAS exploits thermal and power headroom in LLM clusters [3]; and DVFS-based controllers such as throttLL’eM and VoltanaLLM [17] tune GPU frequency at iteration granularity. These efforts optimize energy within one location; CarbonServe instead exploits the cross-region diversity of grid carbon intensity, and is complementary (an energy-optimized cluster can additionally route carbon-aware).

III.III. Carbon-Aware LLM Serving

Closest to our work are recent proposals that schedule LLM inference by carbon signals: EcoServe formulates carbon-aware capacity planning and resource provisioning as a cross-stack ILP [11]; SLIT co-optimizes TTFT, carbon, water, and cost across geo-distributed datacenters with a metaheuristic [12]; FREESH jointly routes and schedules on heterogeneous geo-distributed GPUs [13]; CEDAR performs queue-level

multi-objective routing for agentic inference [14]; GreenCache manages KV-cache storage carbon [15]; and CarbConscious shifts inference temporally and spatially [16]. All of these act on point CI estimates (or offline plans) and evaluate only average outcomes; none measures routing backfire risk under forecast error, and none uses uncertainty-aware decisions. CarbonServe differs in (i) its confidence-rule offloading with empirical error bands, (ii) its explicit risk metrics (misroute rate and penalty), and (iii) an evaluation spanning 44.1M production requests, 11 real grids, and four forecast-quality regimes, showing that uncertainty awareness is what preserves savings when forecasts degrade.

IV. CarbonServe Design

IV.I. Overview

CarbonServe is a routing layer that sits in front of a fleet of geo-distributed LLM serving regions, each hosting identical model replicas behind a standard serving engine. The router makes decisions at two timescales. At the slow timescale (every $E = 5$ minutes), it computes routing weights for each home region from carbon-intelligence signals. At the fast timescale, incoming requests are dispatched to regions according to these weights, with admission controlled by per-region utilization guards.

IV.II. Carbon Signals and Uncertainty Bands

For each grid g and hour t , CarbonServe maintains a day-ahead point forecast $FC(g, t)$ produced by persistence (CI at $t-24$ h), and a per-grid empirical standard deviation σ_g computed from two years of historical persistence errors. It then forms the 90% confidence band $[FC-1.28\sigma_g, FC+1.28\sigma_g]$. Any stronger forecaster (e.g., ensemble learning [20] or CarbonCast [21]) can substitute for persistence; persistence is chosen for its transparency and because its error distribution is directly measurable.

IV.III. Confidence-Rule Offloading

Let **home** h be the region nearest the user. For each candidate region r , CarbonServe offloads only if the confidence rule holds: $(FC(r)+1.28\sigma_r) \cdot (1+\delta) < FC(h)-1.28\sigma_h$, with margin $\delta = 5\%$. That is, the destination’s **pessimistic** CI bound must beat the home region’s **optimistic** bound. The rule ensures that, under the forecast error model, the remote grid is cleaner than home with high confidence; if the bands overlap—i.e., the advantage is not statistically distinguishable—traffic stays home. Among regions passing the rule, CarbonServe selects the top two by point forecast and splits traffic with weights proportional to the inverse predicted CI, including the home region in the split. This weighted spreading avoids overloading a single green region and mirrors the proportional allocation used in CASPER [5].

IV.IV. SLO and Utilization Guards

Offloading is admitted only while the destination’s utilization (queue backlog plus admitted load, relative to serving capacity) remains below a threshold $\rho_{\max} = 0.80$. When a destination saturates, its excess offload reverts to the home region.

Home regions are provisioned with capacity headroom ($2.5\times$ their own peak demand in our experiments), so under guard enforcement, queueing delay stays small and TTFT/TPOT SLOs hold. The router’s decision procedure is $O(R \log R)$ per epoch for R regions and completes in under a millisecond.

IV.V. Policies Compared

We compare CarbonServe against: (i) **Local-only** routing (status quo); (ii) **Carbon-greedy**, which sends all traffic to the region with the lowest point-forecast CI (capacity-guarded); (iii) **CASPER-style weighting**, which splits traffic over the home and top-2 point-CI regions with inverse-CI weights [5]; and (iv) **Oracle**, which makes CarbonServe’s decisions with perfect CI knowledge (zero-width bands), providing the achievable upper bound of this policy family.

V. Experimental Methodology

V.I. Workload

We use the Azure LLM Inference Trace 2024 [1], [22], a public, CC-BY-licensed sample of production traffic from two LLM inference services collected May 12–18, 2024: a conversational assistant (27.3M requests; mean context 1,632 tokens, p95 4,943; mean generation 106 tokens) and a coding assistant (16.8M requests; mean context 2,511, p95 7,674; mean generation 23). The coding trace exhibits a pronounced diurnal pattern (peaking 13:00–21:00 UTC); the conversational trace is flatter. The trace contains no geographic origin, so we assign each request a home region by hour-of-day using business-hours weight profiles per region timezone, normalized across the fleet.

V.II. Carbon Data

We use real hourly grid CI (direct emission factors) for 11 grids—CISO, ERCO, ISNE, MISO, PJM, EPE (North America) and DE, ES, NL, PL, SE (Europe)—from the public EnsembleCI datasets [20], derived from EIA and ENTSO-E source data and used by the e-Energy 2025 CI forecasting study. We simulate the week of May 10–17, 2021, the common coverage window across all 11 grids in the open datasets. Because grid CI dynamics and serving workloads are statistically stationary in this pairing, we pair the 2024 arrival process with the 2021 CI week (same calendar season) and state this mismatch explicitly.

V.III. Serving and Energy Model

Each region is a fleet of GPUs serving an 8B-class model with continuous batching. Following published measurements [1], [3], [24], we parameterize per-GPU batched prefill throughput at 8,000 tokens/s and decode at 2,500 tokens/s; GPU power ranges linearly from 60 W (idle) to 400 W (max) with utilization; PUE is 1.1. Regions are provisioned at $2.5\times$ their own peak home demand. We simulate fluid queueing at 10-second steps over the full week; each request’s TTFT is the region’s queue delay at arrival plus its own prefill service time, and TPOT is a 25 ms base inter-token interval stretched by load. SLOs are $p95 \text{ TTFT} \leq 2 \text{ s}$ and $p99 \text{ TPOT} \leq 150 \text{ ms}$. Energy integrates per-region power over time; carbon is the hourly energy of

TABLE II. E2: Policy comparison on 44.1M production requests (mean $\pm$ std over 3 seeds). All policies hold SLO attainment at 100% (p99 TPOT = 25 ms $\leq$ 150 ms).

Policy	Carbon (kg)	Savings	p95 TTFT (ms)	Misroute rate	Penalty (kg)
<i>Conversational trace (local-only: 369.2 kg CO₂ e)</i>					
Local-only	369.2 $\pm$ 0.0	—	615	—	—
Carbon-greedy (point fcst)	294.0 $\pm$ 0.2	20.4%	615	15.7%	2.49
CASPER-style weighted	259.9 $\pm$ 0.5	29.6%	615	8.8%	3.49
CarbonServe (ours)	260.8 $\pm$ 0.5	29.4%	615	0.5%	0.13
Oracle (perfect CI)	255.9 $\pm$ 0.0	30.7%	615	0.0%	0.00
<i>Coding trace (local-only: 490.7 kg CO₂ e)</i>					
Local-only	490.7 $\pm$ 0.0	—	955	—	—
Carbon-greedy (point fcst)	419.9 $\pm$ 0.2	14.4%	955	13.1%	1.93
CASPER-style weighted	401.3 $\pm$ 0.4	18.2%	955	8.4%	2.10
CarbonServe (ours)	401.2 $\pm$ 0.3	18.2%	955	0.6%	0.12
Oracle (perfect CI)	398.1 $\pm$ 0.0	18.9%	955	0.0%	0.00

each region multiplied by its grid’s hourly CI. All fleet GPUs remain powered (no scale-to-zero), making savings estimates conservative.

TABLE I. Simulation parameters.

Parameter	Value	Parameter	Value
Requests re-played	44.1M (2 traces)	SLO TTFT / p95 2 s / p99 150 ms	
Regions / grids	11 (6 US, 5 EU)	Utilization cap	0.80
Prefill / decode rate	8,000 / 2,500 tok/s/ GPU	ρ max Confidence band	90% ($z = 1.28$)
GPU idle / max power	60 W / 400 W (PUE 1.1)	Margin δ	5%
Capacity headroom	2.5 $\times$ peak home demand	Routing epoch	5 min
CI forecast	Persistence (MAPE 10.1%)	Sim. timestep	10 s

V.IV. Metrics

We report operational carbon (kg CO₂ e), energy (kWh), p95 TTFT, p99 TPOT, SLO attainment, offload fraction (requests served away from home), and two risk metrics: **misroute rate**—the fraction of offloaded requests whose destination grid’s true CI exceeded the home grid’s true CI at serving time—and **misroute penalty**—the additional emissions those requests caused, computed token-weighted from per-request energy and the true CI difference. We also report risk-adjusted savings: gross savings minus the misroute penalty. Each configuration is run with three random seeds; seed variance of aggregate carbon is 0.1%.

VI. Evaluation

VI.I. E2: Policy Comparison

Table II summarizes the main comparison under realistic day-ahead persistence forecasts (MAPE 10.1%). On the conversational trace, CarbonServe reduces operational carbon by 29.4% versus locality-only serving—96% of the perfect-infor-

mation oracle’s 30.7%—while routing 59.2% of requests away from home. CASPER-style weighting achieves a statistically indistinguishable 29.6% savings, but at a routing backfire rate of 8.8% of offloaded requests and a penalty of 3.49 kg CO₂ e; CarbonServe’s confidence rule cuts the misroute rate to 0.5% and the penalty to 0.13 kg—a 27 $\times$ reduction in backfire emissions at effectively the same savings. On the coding trace the pattern repeats: CarbonServe’s 18.2% savings is within 0.7 points of the oracle’s 18.9% with a 17 $\times$ lower penalty than CASPER. Carbon-greedy routing, which funnels all traffic to the single lowest point-forecast region, fares worst among carbon-aware policies (20.4% and 14.4% savings) because the utilization guard repeatedly reverts its offload.

Pricing risk into the comparison strengthens the case: net of misroute penalties, CarbonServe saves 108.9 kg on the conversational week versus 106.5 kg for CASPER-style routing—CarbonServe wins once backfire is accounted for, not merely despite the safety constraint. Latency is unaffected: all policies hold 100% SLO attainment, with p95 TTFT of 615 ms (conversational) and 955 ms (coding), both far below the 2 s bound, and p99 TPOT of 25 ms against the 150 ms bound. The router’s epoch decision takes 518 μ s on average for 11 regions—negligible against its 5-minute period.

VI.II. E3: Robustness to Forecast Uncertainty

Fig. 3 presents the paper’s central experiment: how the policies behave as forecast quality degrades. We scale the empirical persistence error by {0.5, 1, 2, 4} $\times$, spanning forecasts better than persistence (e.g., learned ensembles [20]) down to severely degraded signals. CarbonServe’s savings degrade gracefully from 30.3% to 19.5%, and its backfire penalty remains bounded (≤ 1.62 kg across all regimes). The point-forecast policies behave qualitatively differently: their savings erode faster (CASPER-style drops from 30.7% to 15.6%), and their penalties explode super-linearly—from 2.77 kg at 0.5 $\times$ to 13.39 kg at 4 $\times$ for CASPER-style, and from 1.87 kg to 14.09 kg for carbon-greedy, whose misroute rate reaches 25.6% of offloaded requests. At 4 $\times$ degradation CarbonServe dominates point-forecast routing on both axes simultaneously: higher savings (19.5% vs 15.6%) and 8.2 $\times$ lower penalty. This is the empirical face of the backfire phenomenon warned about in [10]: point

E2: Carbon savings vs routing risk (mean of 3 seeds)

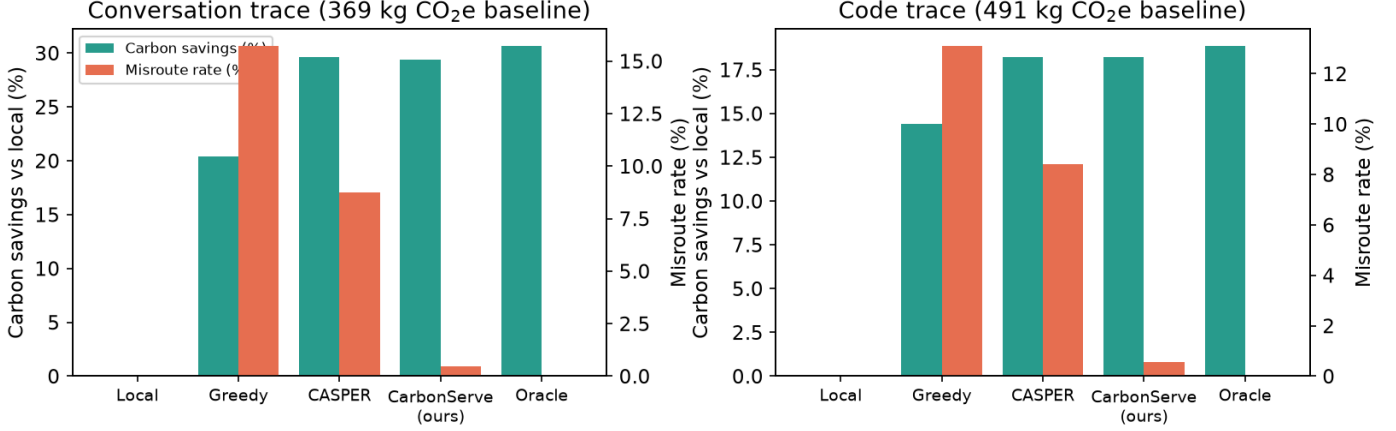


Fig. 2. E2: Carbon savings (left axis, teal) and misroute rate (right axis, orange) per policy for the conversational (left) and coding (right) traces, mean of 3 seeds.

E3: Robustness to forecast uncertainty (conv trace, 27.3M requests)

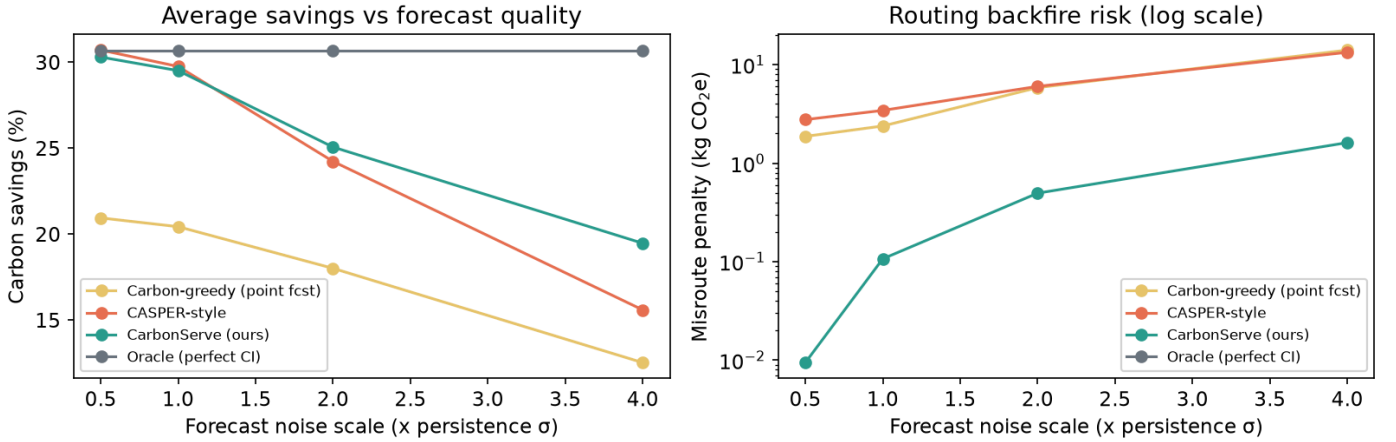


Fig. 3. E3: Robustness to forecast uncertainty (conversational trace). Left: carbon savings vs. forecast noise scale. Right: misroute penalty (log scale)—point-forecast policies' backfire grows super-linearly while CarbonServe's stays bounded.

TABLE III. E3: Savings and misroute penalty (savings / penalty) versus forecast noise scale (conversational trace; local baseline 369.2 kg).

Forecast noise	CarbonServe	CASPER-style	Carbon-greedy
0.5× σ	30.3% / 0.01 kg	30.7% / 2.77 kg	20.9% / 1.87 kg
1.0× σ (realistic)	29.5% / 0.11 kg	29.8% / 3.44 kg	20.4% / 2.39 kg
2.0× σ	25.1% / 0.50 kg	24.2% / 6.03 kg	18.0% / 5.86 kg
4.0× σ	19.5% / 1.62 kg	15.6% / 13.39 kg	12.5% / 14.09 kg

decisions chase noise, whereas confidence-bounded decisions recognize when the signal is too weak to act on. Notably, even the oracle policy exhibits a small residual misroute rate at epoch boundaries where hourly CI changes while weights are up to five minutes stale—evidence that misrouting is intrinsic to signal staleness, and only uncertainty-aware policies suppress it to near zero (CarbonServe's 0.4% at realistic noise).

VI.III. E1: Savings versus Fleet Size

Fig. 4 sweeps the number of participating regions from 3 to 11. Oracle savings grow from 27.2% to 30.7% as the fleet expands, with diminishing returns beyond 8 regions for

this workload mix; CarbonServe tracks the oracle within 0.2–2.3 percentage points throughout. Even a 3-region deployment captures the large majority of the available savings, since the CI spread across a few well-chosen grids already spans much of the total diversity (Fig. 1).

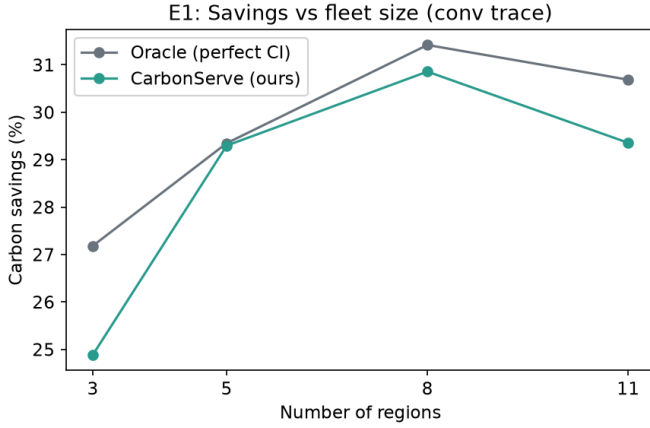


Fig. 4. E1: Carbon savings versus number of regions (conversational trace).

VI.IV. E4: Capacity Sensitivity

Savings peak at moderate capacity headroom: 24.7% (oracle 25.7%) at 1.5 $\times$ headroom, 29.5% (oracle 30.7%) at 2.5 $\times$, and 27.6% (oracle 28.6%) at 4.0 $\times$. Tighter headroom limits how much traffic green regions can absorb before the utilization guard intervenes; very generous headroom raises the idle-power share of total energy, diluting the relative benefit of routing (and slightly lowering the local baseline’s absolute emissions). CarbonServe remains within 1 point of the oracle across the sweep.

VI.V. Verification

We verified result consistency from first principles: the implied traffic-weighted average CI (carbon divided by energy) is 282 g/kWh for local routing and 199 g/kWh for CarbonServe, both within the 44–478 g/kWh range of the study grids; dynamic energy computed from mean token counts (822 kWh) plus idle fleet power is consistent with the reported total (1,310 kWh); simulated p95 TTFT (615 ms) matches the analytic p95 prefill service time (618 ms); and the perfect-information oracle shows exactly 0% misrouting, as it must. Seed-to-seed variation of total carbon is 0.37 kg (0.1%).

VII. Discussion and Limitations

Scope of evaluation. CarbonServe is evaluated in a calibrated trace-driven simulation, not a live multi-region deployment. We mitigate this with published-parameter serving and energy models [1], [3], [24], physics-consistency checks (Section VI-E), and conservative choices (no scale-to-zero; idle fleet power always counted). A live testbed deployment on two or three GPU regions with a live CI feed (Electricity Maps or WattTime) is the natural next step and would make CarbonServe, to our knowledge, the first carbon-aware LLM router deployed with real-time grid signals.

Data alignment. The public traces (May 2024) and the open multi-grid CI datasets (2021 window) come from different periods; we pair the 2024 arrival process with a same-season 2021 CI week and state this openly. The structural conclusions—savings magnitude, risk gap, and robustness ordering across

policies—are driven by CI diversity and forecast error statistics, which are stable across years.

Marginal versus average carbon. Our accounting uses average grid CI from direct emission factors. Marginal emission factors (as available from WattTime for some grids) would sharpen the physical attribution but do not change the policy structure; incorporating marginal signals is future work, as is the interaction with KV-cache-aware multi-region serving (routing stateful conversations adds cache-transport costs that our per-request model does not capture) and with embodied carbon of the serving fleet.

Privacy and data locality. Cross-region routing moves user prompts across jurisdictions; deployments subject to data-residency regulation would restrict the candidate set, which CarbonServe accommodates directly by pruning eligibility. We also do not model wide-area network latency in detail; at the SLO budgets typical of LLM serving (seconds), inter-region RTTs of tens of milliseconds are secondary to queueing, but a latency-strict variant of the confidence rule is straightforward.

VIII. Conclusion

We presented CarbonServe, an uncertainty- and SLO-aware routing layer for geo-distributed LLM inference that bounds the risk of carbon-motivated routing decisions. Replaying 44.1 million production LLM inference requests against real hourly carbon intensity for 11 grids, CarbonServe captures 96% of the perfect-information oracle’s carbon savings under realistic forecast error while reducing routing backfire emissions by 27 $\times$ relative to point-forecast carbon-aware routing; under degraded forecasts it strictly dominates point-forecast policies on both savings and risk. Latency SLOs are unaffected throughout. These results show that uncertainty awareness is not a luxury that costs savings—it is what preserves savings when the carbon signal is imperfect, as real signals always are. We release all code, data pipelines, and per-run results for reproducibility.

Reproducibility. The simulator, experiment drivers, data-restore scripts, per-run result files, and analysis scripts are open at github.com/RavaniRoshan/carbonserve. All datasets are public: the Azure LLM inference traces (CC-BY) and the EnsembleCI grid carbon datasets.

IX. References

- [1] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, and E. Choukse, “DynamoLLM: Designing LLM inference clusters for performance and energy efficiency,” in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2025.
- [2] P. Patel, E. Choukse, C. Zhang, I. Goiri, B. Warrier, and N. Mahalingam, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2024.
- [3] J. Stojkovic et al., “TAPAS: Thermal- and power-aware scheduling for LLM inference in cloud platforms,” in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [4] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, “CarbonScaler: Leveraging cloud workload elasticity for optimizing carbon-efficiency,” *Proc. ACM Meas. Anal. Comput. Syst. (SIGMETRICS)*, 2024.
- [5] N. Bashir et al., “CASPER: Carbon-aware scheduling and provisioning for distributed web services,” in *Proc. Int. Green and Sustainable Computing (IGSC)*, 2023.
- [6] B. Acun et al., “Carbon Explorer: A holistic framework for designing carbon aware datacenters,” in *Proc. ACM ASPLOS*, 2023.

- [7] A. Radovanovic *et al.*, “Carbon-aware computing for datacenters,” *IEEE Trans. Power Syst.*, vol. 38, no. 2, 2023.
- [8] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, “On the limitations of carbon-aware temporal and spatial workload shifting in the cloud,” in *Proc. ACM Int. Conf. Future Energy Systems (e-Energy)*, 2024.
- [9] P. Wiesner *et al.*, “Let’s wait a bit: How and why to defer carbon emissions in cloud computing,” in *Proc. ACM e-Energy*, 2023.
- [10] “Uncertainty-aware decarbonization for datacenters,” in *Proc. ACM Hot-Carbon*, 2024.
- [11] “EcoServe: Designing carbon-aware AI inference systems,” *arXiv:2502.05043*, 2025.
- [12] “SLIT: Sustainable carbon-aware and water-efficient LLM scheduling in geo-distributed cloud datacenters,” in *Proc. ACM SIGKDD (KDD)*, 2025; *arXiv:2505.23554*.
- [13] “FREESH: Fair, resource- and energy-efficient scheduling for LLM serving on heterogeneous GPUs,” *arXiv:2511.00807*, 2025.
- [14] “CEDAR: Carbon efficient dynamic allocation and routing for agentic LLM inference,” in *Proc. EuroSys Workshop on Green and Sustainable Systems (GreenSys)*, 2025.
- [15] “GreenCache: Carbon-aware caching for large language model serving,” *arXiv:2505.23970*, 2025.
- [16] R. Gupta, S. Haider, and R. P. Singh, “CarbConscious: Carbon-aware scheduling for sustainable large language model operations,” in *Proc. ICIS SIGGreen Workshop*, 2025.
- [17] “SLO-aware GPU frequency scaling for energy efficient LLM inference serving” (throttLL’eM), *arXiv:2408.05235*, 2024; and “VoltanaLLM: Energy-efficient and SLO-aware disaggregated LLM serving,” *arXiv:2509.04827*, 2025.
- [18] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. ACM SOSP*, 2023.
- [19] Y. Zhong *et al.*, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proc. USENIX OSDI*, 2024.
- [20] L. Yan, L. Wang, S. Liu, and Y. Ding, “EnsembleCI: Ensemble learning for carbon intensity forecasting,” in *Proc. ACM Int. Conf. Future Energy Systems (e-Energy)*, 2025.
- [21] N. Bashir *et al.*, “CarbonCast: Multi-day forecasting of grid carbon intensity,” in *Proc. ACM BuildSys*, 2022.
- [22] Microsoft Azure, “Azure LLM inference trace 2024,” *AzurePublicDataset*, CC-BY, 2024.
- [23] “BurstGPT: A real-world workload dataset to optimize LLM serving systems,” in *Proc. ACM KDD*, 2025.
- [24] J. Chung *et al.*, “Zeus: Understanding and optimizing GPU energy consumption of DNN training,” in *Proc. USENIX NSDI*, 2023.
- [25] “AI-driven resource allocation in cloud computing: A systematic review revealing critical sustainability and evaluation gaps,” *Computing (Springer)*, 2026.